EXPERIMENT

Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python.

Name : S.Nagulan

Reg No : 1924260216

(This experiment is divided into 5 problems)

Problem 1: Fruit Classification Data Set

Aim

To write a Python program that implements the K-Nearest Neighbours (K-NN) algorithm with K = 3, using the Fruit Classification Data Set to decide whether a fruit is an Apple or an Orange from its weight and size, and to classify a new sample.

Procedure

1. Store the training examples (2 features: Weight (g), Size (cm); class: Fruit), the test examples and the new sample as lists, and choose the value of K (3).
2. Calculate the Euclidean distance between the new sample and every training example: d = square root of the sum of (x1 - y1)^2 + (x2 - y2)^2 + ...
3. Sort the training examples in ascending order of distance and select the K nearest neighbours.
4. Count the class labels of the K nearest neighbours (majority voting) and assign the most frequent class to the new sample.
5. Repeat the same steps for every test example, compare the predicted class with the actual class and count the correct predictions.
6. Calculate the accuracy = (correct predictions / total test examples) x 100 and display the predicted class and the accuracy.

Coding (Python)

```
# K-Nearest Neighbours (K-NN) - Fruit Classification Data Set
import math
from collections import Counter

K = 3

# Training data: [Weight (g), Size (cm), Class]
train = [
    [150, 7.0, 'Apple'],
    [160, 7.5, 'Apple'],
    [170, 7.8, 'Apple'],
    [140, 6.8, 'Apple'],
    [120, 5.5, 'Orange'],
    [110, 5.0, 'Orange'],
    [100, 4.8, 'Orange'],
    [125, 5.8, 'Orange'],
]

# Test data (known classes) to measure the accuracy
test = [
    [165, 7.6, 'Apple'],
    [115, 5.2, 'Orange'],
    [145, 6.9, 'Apple'],
]

# New sample whose class is to be predicted
new_sample = [130, 6.0]
def euclidean(a, b):
```

```
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def knn_predict(train, x, k):
    # distance of x from every training example
    distances = []
    for row in train:
        distances.append((euclidean(row[:-1], x), row[-1]))
    distances.sort(key=lambda d: d[0])       # sort in ascending order
    neighbours = distances[:k]               # k nearest neighbours
    votes = Counter(label for _, label in neighbours)
    return votes.most_common(1)[0][0], neighbours, votes

# Step 1: predict the class of the new sample
label, neighbours, votes = knn_predict(train, new_sample, K)
print("Value of K :", K)
print("New sample :", new_sample)
print("\nK nearest neighbours (distance | class):")
for d, c in neighbours:
    print("   ", round(d, 3), "|", c)
print("\nVotes :", dict(votes))
print("Predicted class of the new sample :", label)

# Step 2: test the classifier on the test data
print("\nTesting the classifier (sample | actual | predicted):")
correct = 0
for row in test:
    pred = knn_predict(train, row[:-1], K)[0]
    print("   ", row[:-1], "|", row[-1], "|", pred)
    if pred == row[-1]:
        correct += 1
accuracy = correct / len(test) * 100
print("\nAccuracy :", round(accuracy, 2), "%")
```

Input

Training data set (8 examples, K = 3):

Weight (g)	Size (cm)	Fruit
150	7.0	Apple
160	7.5	Apple
170	7.8	Apple
140	6.8	Apple
120	5.5	Orange
110	5.0	Orange
100	4.8	Orange
125	5.8	Orange

Test data set (used to find the accuracy):

Weight (g)	Size (cm)	Fruit
165	7.6	Apple
115	5.2	Orange
145	6.9	Apple

New sample to be classified:

Weight (g)	Size (cm)
130	6.0

Output

```
Value of K : 3
New sample : [130, 6.0]

K nearest neighbours (distance | class):
   5.004 | Orange
   10.012 | Orange
   10.032 | Apple

Votes : {'Orange': 2, 'Apple': 1}
Predicted class of the new sample : Orange

Testing the classifier (sample | actual | predicted):
   [165, 7.6] | Apple | Apple
   [115, 5.2] | Orange | Orange
   [145, 6.9] | Apple | Apple

Accuracy : 100.0 %
```

Result

Thus the K-Nearest Neighbours algorithm was implemented in Python successfully. For K = 3, the new sample was classified as **Fruit = Orange** and the classifier achieved an accuracy of **100.0 %** on the test data.

Problem 2: Student Pass / Fail Data Set

Aim

To write a Python program that implements the K-Nearest Neighbours (K-NN) algorithm with K = 3, using the Student Pass / Fail Data Set to decide whether a student passes or fails from study hours and attendance, and to classify a new sample.

Procedure

1. Store the training examples (2 features: Hours Studied, Attendance %; class: Result), the test examples and the new sample as lists, and choose the value of K (3).
2. Calculate the Euclidean distance between the new sample and every training example: d = square root of the sum of (x1 - y1)^2 + (x2 - y2)^2 + ...
3. Sort the training examples in ascending order of distance and select the K nearest neighbours.
4. Count the class labels of the K nearest neighbours (majority voting) and assign the most frequent class to the new sample.
5. Repeat the same steps for every test example, compare the predicted class with the actual class and count the correct predictions.
6. Calculate the accuracy = (correct predictions / total test examples) x 100 and display the predicted class and the accuracy.

Coding (Python)

```
# K-Nearest Neighbours (K-NN) - Student Pass / Fail Data Set
import math
from collections import Counter

K = 3

# Training data: [Hours Studied, Attendance %, Class]
train = [
    [1, 40, 'Fail'],
    [2, 50, 'Fail'],
    [3, 45, 'Fail'],
    [4, 60, 'Fail'],
    [5, 55, 'Fail'],
    [6, 80, 'Pass'],
    [7, 75, 'Pass'],
    [8, 90, 'Pass'],
    [9, 85, 'Pass'],
    [10, 95, 'Pass'],
]

# Test data (known classes) to measure the accuracy
test = [
    [2, 45, 'Fail'],
    [9, 92, 'Pass'],
    [7, 70, 'Pass'],
    [4, 50, 'Fail'],
]
# New sample whose class is to be predicted
```

```
new_sample = [6, 70]

def euclidean(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def knn_predict(train, x, k):
    # distance of x from every training example
    distances = []
    for row in train:
        distances.append((euclidean(row[:-1], x), row[-1]))
    distances.sort(key=lambda d: d[0])       # sort in ascending order
    neighbours = distances[:k]               # k nearest neighbours
    votes = Counter(label for _, label in neighbours)
    return votes.most_common(1)[0][0], neighbours, votes

# Step 1: predict the class of the new sample
label, neighbours, votes = knn_predict(train, new_sample, K)
print("Value of K :", K)
print("New sample :", new_sample)
print("\nK nearest neighbours (distance | class):")
for d, c in neighbours:
    print("   ", round(d, 3), "|", c)
print("\nVotes :", dict(votes))
print("Predicted class of the new sample :", label)

# Step 2: test the classifier on the test data
print("\nTesting the classifier (sample | actual | predicted):")
correct = 0
for row in test:
    pred = knn_predict(train, row[:-1], K)[0]
    print("   ", row[:-1], "|", row[-1], "|", pred)
    if pred == row[-1]:
        correct += 1
accuracy = correct / len(test) * 100
print("\nAccuracy :", round(accuracy, 2), "%")
```

Input

Training data set (10 examples, K = 3):

Hours Studied	Attendance %	Result
1	40	Fail
2	50	Fail
3	45	Fail
4	60	Fail
5	55	Fail
6	80	Pass
7	75	Pass
8	90	Pass
9	85	Pass
10	95	Pass

Test data set (used to find the accuracy):

Hours Studied	Attendance %	Result
2	45	Fail

Hours Studied	Attendance %	Result
9	92	Pass
7	70	Pass
4	50	Fail

New sample to be classified:

Hours Studied	Attendance %
6	70

Output

```
Value of K : 3
New sample : [6, 70]

K nearest neighbours (distance | class):
   5.099 | Pass
   10.0 | Pass
   10.198 | Fail

Votes : {'Pass': 2, 'Fail': 1}
Predicted class of the new sample : Pass

Testing the classifier (sample | actual | predicted):
   [2, 45] | Fail | Fail
   [9, 92] | Pass | Pass
   [7, 70] | Pass | Pass
   [4, 50] | Fail | Fail

Accuracy : 100.0 %
```

Result

Thus the K-Nearest Neighbours algorithm was implemented in Python successfully. For K = 3, the new sample was classified as **Result = Pass** and the classifier achieved an accuracy of **100.0 %** on the test data.

Problem 3: Iris Flower Data Set (Sample)

Aim

To write a Python program that implements the K-Nearest Neighbours (K-NN) algorithm with K = 3, using the Iris Flower Data Set (Sample) to decide the species of an iris flower from its sepal and petal measurements, and to classify a new sample.

Procedure

1. Store the training examples (4 features: Sepal Length, Sepal Width, Petal Length, Petal Width; class: Species), the test examples and the new sample as lists, and choose the value of K (3).
2. Calculate the Euclidean distance between the new sample and every training example: d = square root of the sum of (x1 - y1)^2 + (x2 - y2)^2 + ...
3. Sort the training examples in ascending order of distance and select the K nearest neighbours.
4. Count the class labels of the K nearest neighbours (majority voting) and assign the most frequent class to the new sample.
5. Repeat the same steps for every test example, compare the predicted class with the actual class and count the correct predictions.
6. Calculate the accuracy = (correct predictions / total test examples) x 100 and display the predicted class and the accuracy.

Coding (Python)

```
# K-Nearest Neighbours (K-NN) - Iris Flower Data Set (Sample)
import math
from collections import Counter

K = 3

# Training data: [Sepal Length, Sepal Width, Petal Length, Petal Width, Class]
train = [
    [5.1, 3.5, 1.4, 0.2, 'Setosa'],
    [4.9, 3.0, 1.4, 0.2, 'Setosa'],
    [4.7, 3.2, 1.3, 0.2, 'Setosa'],
    [5.0, 3.6, 1.4, 0.2, 'Setosa'],
    [7.0, 3.2, 4.7, 1.4, 'Versicolor'],
    [6.4, 3.2, 4.5, 1.5, 'Versicolor'],
    [6.9, 3.1, 4.9, 1.5, 'Versicolor'],
    [5.5, 2.3, 4.0, 1.3, 'Versicolor'],
    [6.3, 3.3, 6.0, 2.5, 'Virginica'],
    [5.8, 2.7, 5.1, 1.9, 'Virginica'],
    [7.1, 3.0, 5.9, 2.1, 'Virginica'],
    [6.5, 3.0, 5.8, 2.2, 'Virginica'],
]

# Test data (known classes) to measure the accuracy
test = [
    [5.4, 3.9, 1.3, 0.4, 'Setosa'],
    [6.0, 2.9, 4.5, 1.5, 'Versicolor'],
    [6.7, 3.1, 5.6, 2.4, 'Virginica'],
```

```
]

# New sample whose class is to be predicted
new_sample = [5.9, 3.0, 5.1, 1.8]

def euclidean(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def knn_predict(train, x, k):
    # distance of x from every training example
    distances = []
    for row in train:
        distances.append((euclidean(row[:-1], x), row[-1]))
    distances.sort(key=lambda d: d[0])       # sort in ascending order
    neighbours = distances[:k]               # k nearest neighbours
    votes = Counter(label for _, label in neighbours)
    return votes.most_common(1)[0][0], neighbours, votes

# Step 1: predict the class of the new sample
label, neighbours, votes = knn_predict(train, new_sample, K)
print("Value of K :", K)
print("New sample :", new_sample)
print("\nK nearest neighbours (distance | class):")
for d, c in neighbours:
    print("   ", round(d, 3), "|", c)
print("\nVotes :", dict(votes))
print("Predicted class of the new sample :", label)

# Step 2: test the classifier on the test data
print("\nTesting the classifier (sample | actual | predicted):")
correct = 0
for row in test:
    pred = knn_predict(train, row[:-1], K)[0]
    print("   ", row[:-1], "|", row[-1], "|", pred)
    if pred == row[-1]:
        correct += 1
accuracy = correct / len(test) * 100
print("\nAccuracy :", round(accuracy, 2), "%")
```

Input

Training data set (12 examples, K = 3):

Sepal Length	Sepal Width	Petal Length	Petal Width	Species
5.1	3.5	1.4	0.2	Setosa
4.9	3.0	1.4	0.2	Setosa
4.7	3.2	1.3	0.2	Setosa
5.0	3.6	1.4	0.2	Setosa
7.0	3.2	4.7	1.4	Versicolor
6.4	3.2	4.5	1.5	Versicolor
6.9	3.1	4.9	1.5	Versicolor
5.5	2.3	4.0	1.3	Versicolor
6.3	3.3	6.0	2.5	Virginica
5.8	2.7	5.1	1.9	Virginica
7.1	3.0	5.9	2.1	Virginica

Sepal Length	Sepal Width	Petal Length	Petal Width	Species
6.5	3.0	5.8	2.2	Virginica

Test data set (used to find the accuracy):

Sepal Length	Sepal Width	Petal Length	Petal Width	Species
5.4	3.9	1.3	0.4	Setosa
6.0	2.9	4.5	1.5	Versicolor
6.7	3.1	5.6	2.4	Virginica

New sample to be classified:

Sepal Length	Sepal Width	Petal Length	Petal Width
5.9	3.0	5.1	1.8

Output

```
Value of K : 3
New sample : [5.9, 3.0, 5.1, 1.8]

K nearest neighbours (distance | class):
   0.332 | Virginica
   0.86 | Versicolor
   1.005 | Virginica

Votes : {'Virginica': 2, 'Versicolor': 1}
Predicted class of the new sample : Virginica

Testing the classifier (sample | actual | predicted):
   [5.4, 3.9, 1.3, 0.4] | Setosa | Setosa
   [6.0, 2.9, 4.5, 1.5] | Versicolor | Versicolor
   [6.7, 3.1, 5.6, 2.4] | Virginica | Virginica

Accuracy : 100.0 %
```

Result

Thus the K-Nearest Neighbours algorithm was implemented in Python successfully. For K = 3, the new sample was classified as **Species = Virginica** and the classifier achieved an accuracy of **100.0 %** on the test data.

Problem 4: Customer Purchase Data Set

Aim

To write a Python program that implements the K-Nearest Neighbours (K-NN) algorithm with K = 5, using the Customer Purchase Data Set to decide whether a customer purchases a product from age and salary, and to classify a new sample.

Procedure

1. Store the training examples (2 features: Age, Salary (in thousands); class: Purchased), the test examples and the new sample as lists, and choose the value of K (5).
2. Calculate the Euclidean distance between the new sample and every training example: d = square root of the sum of (x1 - y1)^2 + (x2 - y2)^2 + ...
3. Sort the training examples in ascending order of distance and select the K nearest neighbours.
4. Count the class labels of the K nearest neighbours (majority voting) and assign the most frequent class to the new sample.
5. Repeat the same steps for every test example, compare the predicted class with the actual class and count the correct predictions.
6. Calculate the accuracy = (correct predictions / total test examples) x 100 and display the predicted class and the accuracy.

Coding (Python)

```
# K-Nearest Neighbours (K-NN) - Customer Purchase Data Set
import math
from collections import Counter

K = 5

# Training data: [Age, Salary (in thousands), Class]
train = [
    [19, 19, 'No'],
    [21, 35, 'No'],
    [25, 30, 'No'],
    [28, 45, 'No'],
    [30, 40, 'No'],
    [35, 60, 'No'],
    [38, 150, 'Yes'],
    [42, 90, 'Yes'],
    [45, 120, 'Yes'],
    [48, 140, 'Yes'],
    [52, 110, 'Yes'],
    [27, 85, 'Yes'],
]

# Test data (known classes) to measure the accuracy
test = [
    [23, 28, 'No'],
    [46, 130, 'Yes'],
    [32, 50, 'No'],
    [50, 100, 'Yes'],
```

```
]

# New sample whose class is to be predicted
new_sample = [40, 95]

def euclidean(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def knn_predict(train, x, k):
    # distance of x from every training example
    distances = []
    for row in train:
        distances.append((euclidean(row[:-1], x), row[-1]))
    distances.sort(key=lambda d: d[0])       # sort in ascending order
    neighbours = distances[:k]               # k nearest neighbours
    votes = Counter(label for _, label in neighbours)
    return votes.most_common(1)[0][0], neighbours, votes

# Step 1: predict the class of the new sample
label, neighbours, votes = knn_predict(train, new_sample, K)
print("Value of K :", K)
print("New sample :", new_sample)
print("\nK nearest neighbours (distance | class):")
for d, c in neighbours:
    print("   ", round(d, 3), "|", c)
print("\nVotes :", dict(votes))
print("Predicted class of the new sample :", label)

# Step 2: test the classifier on the test data
print("\nTesting the classifier (sample | actual | predicted):")
correct = 0
for row in test:
    pred = knn_predict(train, row[:-1], K)[0]
    print("   ", row[:-1], "|", row[-1], "|", pred)
    if pred == row[-1]:
        correct += 1
accuracy = correct / len(test) * 100
print("\nAccuracy :", round(accuracy, 2), "%")
```

Input

Training data set (12 examples, K = 5):

Age	Salary (in thousands)	Purchased
19	19	No
21	35	No
25	30	No
28	45	No
30	40	No
35	60	No
38	150	Yes
42	90	Yes
45	120	Yes
48	140	Yes
52	110	Yes

Age	Salary (in thousands)	Purchased
27	85	Yes

Test data set (used to find the accuracy):

Age	Salary (in thousands)	Purchased
23	28	No
46	130	Yes
32	50	No
50	100	Yes

New sample to be classified:

Age	Salary (in thousands)
40	95

Output

```
Value of K : 5
New sample : [40, 95]

K nearest neighbours (distance | class):
   5.385 | Yes
   16.401 | Yes
   19.209 | Yes
   25.495 | Yes
   35.355 | No

Votes : {'Yes': 4, 'No': 1}
Predicted class of the new sample : Yes

Testing the classifier (sample | actual | predicted):
   [23, 28] | No | No
   [46, 130] | Yes | Yes
   [32, 50] | No | No
   [50, 100] | Yes | Yes

Accuracy : 100.0 %
```

Result

Thus the K-Nearest Neighbours algorithm was implemented in Python successfully. For K = 5, the new sample was classified as **Purchased = Yes** and the classifier achieved an accuracy of **100.0 %** on the test data.

Problem 5: Diabetes Prediction Data Set

Aim

To write a Python program that implements the K-Nearest Neighbours (K-NN) algorithm with K = 3, using the Diabetes Prediction Data Set to decide whether a patient is diabetic from glucose level and body mass index, and to classify a new sample.

Procedure

1. Store the training examples (2 features: Glucose, BMI; class: Diabetic), the test examples and the new sample as lists, and choose the value of K (3).
2. Calculate the Euclidean distance between the new sample and every training example: d = square root of the sum of (x1 - y1)^2 + (x2 - y2)^2 + ...
3. Sort the training examples in ascending order of distance and select the K nearest neighbours.
4. Count the class labels of the K nearest neighbours (majority voting) and assign the most frequent class to the new sample.
5. Repeat the same steps for every test example, compare the predicted class with the actual class and count the correct predictions.
6. Calculate the accuracy = (correct predictions / total test examples) x 100 and display the predicted class and the accuracy.

Coding (Python)

```
# K-Nearest Neighbours (K-NN) - Diabetes Prediction Data Set
import math
from collections import Counter

K = 3

# Training data: [Glucose, BMI, Class]
train = [
    [85, 22.5, 'No'],
    [90, 24.0, 'No'],
    [95, 26.0, 'No'],
    [100, 25.5, 'No'],
    [88, 23.0, 'No'],
    [150, 34.0, 'Yes'],
    [165, 36.5, 'Yes'],
    [140, 32.0, 'Yes'],
    [155, 35.0, 'Yes'],
    [170, 38.0, 'Yes'],
]

# Test data (known classes) to measure the accuracy
test = [
    [92, 24.5, 'No'],
    [160, 35.5, 'Yes'],
    [145, 33.0, 'Yes'],
    [98, 26.5, 'No'],
]
# New sample whose class is to be predicted
```

```
new_sample = [130, 30.0]

def euclidean(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def knn_predict(train, x, k):
    # distance of x from every training example
    distances = []
    for row in train:
        distances.append((euclidean(row[:-1], x), row[-1]))
    distances.sort(key=lambda d: d[0])       # sort in ascending order
    neighbours = distances[:k]               # k nearest neighbours
    votes = Counter(label for _, label in neighbours)
    return votes.most_common(1)[0][0], neighbours, votes

# Step 1: predict the class of the new sample
label, neighbours, votes = knn_predict(train, new_sample, K)
print("Value of K :", K)
print("New sample :", new_sample)
print("\nK nearest neighbours (distance | class):")
for d, c in neighbours:
    print("   ", round(d, 3), "|", c)
print("\nVotes :", dict(votes))
print("Predicted class of the new sample :", label)

# Step 2: test the classifier on the test data
print("\nTesting the classifier (sample | actual | predicted):")
correct = 0
for row in test:
    pred = knn_predict(train, row[:-1], K)[0]
    print("   ", row[:-1], "|", row[-1], "|", pred)
    if pred == row[-1]:
        correct += 1
accuracy = correct / len(test) * 100
print("\nAccuracy :", round(accuracy, 2), "%")
```

Input

Training data set (10 examples, K = 3):

Glucose	BMI	Diabetic
85	22.5	No
90	24.0	No
95	26.0	No
100	25.5	No
88	23.0	No
150	34.0	Yes
165	36.5	Yes
140	32.0	Yes
155	35.0	Yes
170	38.0	Yes

Test data set (used to find the accuracy):

Glucose	BMI	Diabetic
92	24.5	No

Glucose	BMI	Diabetic
160	35.5	Yes
145	33.0	Yes
98	26.5	No

New sample to be classified:

Glucose	BMI
130	30.0

Output

```
Value of K : 3
New sample : [130, 30.0]

K nearest neighbours (distance | class):
   10.198 | Yes
   20.396 | Yes
   25.495 | Yes

Votes : {'Yes': 3}
Predicted class of the new sample : Yes

Testing the classifier (sample | actual | predicted):
   [92, 24.5] | No | No
   [160, 35.5] | Yes | Yes
   [145, 33.0] | Yes | Yes
   [98, 26.5] | No | No

Accuracy : 100.0 %
```

Result

Thus the K-Nearest Neighbours algorithm was implemented in Python successfully. For K = 3, the new sample was classified as **Diabetic = Yes** and the classifier achieved an accuracy of **100.0 %** on the test data.